

NIGHT WATCH

TECHNISCHE DOKUMENTATION

"There is no escape. There is only dark."

Onur Gündüz & Fabian Bechter

Klasse 1AHIF · HTL Rankweil · Juni 2026

Python 3.12+

pygame 2.6.1

NumPy · Pillow

— 00

Inhaltsverzeichnis



01	Projektbeschreibung	03
02	Spielkonzept & Mechaniken	04
03	Monster-KI & Wegfindung	05
04	Technische Architektur	06
05	Module & Klassen	07
06	KI-Verwendung	08
07	Steuerung & Bedienung	09
08	Reflexion & Fazit	10

— 01

Projektbeschreibung

Nightwatch ist ein zweidimensionales Top-Down-Horror-Spiel, entwickelt als Endabgabe für den POS-Unterricht an der HTL Rankweil. Das Spiel ist vollständig in Python 3 geschrieben und verwendet ausschließlich `pygame 2.6.1`. Raumgrafiken und Level-Elemente werden beim ersten Start prozedural mit Pillow generiert. Spieler- und Monster-Sprites wurden handgezeichnet in **Aseprite** und **Adobe Photoshop 2026**. Sounds werden vollständig mit NumPy erzeugt.

— SPIELZIEL

Der Spieler ist in einem dunklen Haus eingeschlossen und muss **fünf Aufgaben** in verschiedenen Räumen erledigen, bevor er durch den Ausgang entkommen kann. Gleichzeitig patrouilliert ein Monster durch das Haus – wird der Spieler erwischt, ist das Spiel vorbei. Die einzige Lichtquelle ist eine **Taschenlampe mit begrenztem Akku**. Batterien im Level können erst aufgesammelt werden, wenn der Akku unter 15% fällt.

— ENTSTEHUNG & INSPIRATION

Die Idee entstand nach dem Spielen verschiedener Horror-Indie-Spiele. Ziel war zu zeigen, dass man mit reinem Python ohne externe Game-Engine ein echtes Spielerlebnis erzeugen kann. Das Konzept orientiert sich an klassischen Escape-Room-Spielen kombiniert mit Stealth-Elementen: Schleichen, Lichtmanagement und das Vermeiden des Monsters sind die Kernmechaniken.

— TECHNOLOGIEN IM ÜBERBLICK

BIBLIOTHEK	VERWENDUNGSZWECK
Python 3.12+	Programmiersprache
pygame 2.6.1	2D-Grafik, Sound, Eingabe, Spielschleife
NumPy ≥ 1.26	Prozedurale Soundgenerierung (Ambient-Loop, Schritte)
Pillow ≥ 10.0	Prozedurale Generierung von Raumelementen & Level-Grafiken

KEINE EXTERNEN ASSETS

Sprites handgezeichnet in Aseprite & Photoshop 2026. Nur Level-Grafiken prozedural.

BFS-WEGFINDUNG

Vollständige KI-Feind-Navigation via Breadth-First Search.

5 AUFGABENTYPEN

Jede Aufgabe mit eigener Interaktionslogik und Schlüssel-Spawn.

— 02

Spielkonzept & Mechaniken

02

— DIE 5 AUFGABEN

Im Haus sind fünf Aufgaben verteilt. Jede abgeschlossene Aufgabe spawnt einen Schlüssel. Erst wenn alle fünf Schlüssel gesammelt sind, öffnet sich der Ausgang.

01 Alle 5 Kerzen mit [E] anzünden

02 Drei Lichtschalter der Reihe nach umlegen

03 Eine Kiste zur Zielmarkierung tragen

04 Dreistelligen Code vom Hinweiszettel eingeben

— TASCHENLAMPE & AKKU

Die Taschenlampe ist die einzige Lichtquelle. Der Akku leert sich kontinuierlich. Versteckte Batterien können erst aufgesammelt werden, wenn der Akku unter **15%** sinkt – eine bewusste Designentscheidung zur Spannungserzeugung. Das Monster sieht im Dunklen schlechter, aber ausschalten ist trotzdem riskant.

— DASH-MECHANIK

Der Dash ermöglicht einen kurzen Ausweich-Sprint mit **60 Sekunden Cooldown** und muss strategisch eingesetzt werden. Das Monster wird dabei kurz betäubt. Während des Dashes hinterlässt der Spieler einen blauen Leuchteffekt.

— SCHWIERIGKEITSGRADE

SEHR EINFACH

Sehr langsames Monster,
weite Toleranz

EINFACH

Langsames Monster,
kurze Sichtweite

MITTEL

Standard-Balancierung,
schnellere Reaktion

SCHWER

Schnelleres Monster,
kaum Toleranz

— GERÄUSCHSYSTEM

Teppichböden dämpfen Schrittgeräusche, Parkett und Fliesen leiten sie weiter. Das Monster reagiert erst ab einem Mindest-Lärmpegel und einer maximalen Entfernung – der Spieler muss also nicht immer schleichen, in der Nähe des Monsters aber schon.

— 03

Monster-KI & Wegfindung

— ZUSTANDSMASCHINE

Das Monster operiert über eine Zustandsmaschine mit drei Zuständen. Die Übergänge hängen von Sichtweite, Lärmpegel und Bodentyp ab.

PATROL

Vordefinierte Routen durch das Haus abpatrouillieren

ALERT

Geräusch gehört – Bewegung zum letzten bekannten Standort

HUNT

Spieler direkt gesehen – aktive Verfolgung, Herzschlag beschleunigt

— BFS-WEGFINDUNG

Das Spielfeld wird in ein Grid aus **20×20-Pixel-Zellen** aufgeteilt. Der Breadth-First Search-Algorithmus findet den kürzesten Weg vom Monster zum Spieler. Wände und Hindernisse werden als nicht begehbar markiert.

KI CODE — CLAUDE OPUS 4.5

```
def bfs_pfad(grid, start, ziel):
    queue = deque([[start]])
    visited = {start}

    while queue:
        pfad = queue.popleft()
        knoten = pfad[-1]
        if knoten == ziel:
            return pfad
        for nachbar in get_nachbarn(grid, knoten):
            if nachbar not in visited:
                visited.add(nachbar)
                queue.append(pfad + [nachbar])
    return []
```

— SICHT & GERÄUSCHERKENNUNG

Das Monster erkennt den Spieler über zwei unabhängige Kanäle: **Sicht** (innerhalb der Sichtweite) und **Geräusch** (Lärmpegel multipliziert mit bodenabhängigem Faktor ergibt maximale Hörweite). Beide Checks können unabhängig einen Zustandswechsel auslösen.

— JUMPSCARE-SYSTEM

- **Fast-Miss:** Kurzer visueller Schreck bei fast erwischt
- **Raum-Jumpscare:** Schlafzimmer bei aufmerksamem Monster
- **Screen-Shake:** Läuft über eigenen ShakeOffset-Buffer

— 04

Technische Architektur

— PROJEKTSTRUKTUR

Das Projekt ist in zwei Python-Pakete aufgeteilt: `game/` enthält alle Spielmodule, `game_variables/` die globalen Konstanten. Einstiegspunkt ist `src/main.py`.

```
nightwatch-abgabe/
├─ requirements.txt
├─ licenses.txt
├─ src/
│   ├── main.py          # Hauptloop, GameState, ZoomCamera, PauseMenu
│   ├── game_variables/
│   │   ├── __init__.py
│   │   └─ game_variables.py # GV-Klasse: alle Konstanten
│   └─ game/
│
│   ├── level.py         # Räume, Korridore, Türen, Exit
│   ├── player.py        # Spieler: Bewegung, Dash, Taschenlampe
│   ├── monster.py       # Monster-KI: State Machine + BFS
│   ├── items.py         # Aufgaben, Schlüssel, Batterie-Pickups
│   ├── lighting.py      # Taschenlampe, Filmkorn, Vignette
│   ├── hud.py           # HUD: Akku, Schlüssel, Hinweise
│   ├── jumpscare.py     # Jumpscare-Effekte & Screen-Shake
│   ├── highscore.py     # Highscore speichern & anzeigen
│   └─ sprites.py        # draw_blood_splatter, Ketten, ...
└─ assets/
    ├── player_sheet.png # Aseprite + Adobe Photoshop 2026
    └─ monster_sheet.png # Aseprite + Adobe Photoshop 2026
```

— GAMEVARIABLES-MUSTER

Alle globalen Konstanten (Auflösung, Farben, Geschwindigkeiten, Schwierigkeitswerte) sind in der Klasse `GameVariables` zentralisiert. Das Muster `GV = GameVariables()` wird in jedem Modul verwendet, um Tipparbeit zu sparen. Die `Init`-Methode initialisiert `pygame` und setzt die Schriftarten.

```
# Jedes Modul importiert so:
from game_variables import GameVariables
GV = GameVariables()

# Verwendung überall:
GV.SCREEN_WIDTH, GV.MONSTER_SPEED, GV.FLASHLIGHT_DRAIN
```

— 05

Module & Klassen

Player

Bewegung via WASD / Pfeiltasten, Taschenlampe per Maus ausrichten, Dash-Mechanik (60s Cooldown), Schleichen mit Shift, Wand- und Türkollision.

Monster

Zustandsmaschine PATROL → ALERT → HUNT, BFS-Wegfindung auf 20×20px Grid, Sicht- und Geräuscherkennung, Geschwindigkeit nach Schwierigkeitsgrad.

ItemManager

Verwaltung aller 5 Aufgaben, Schlüssel-Spawning nach Abschluss, Batterie-Pickups (nur bei Akku <15%), Exit-Tür bei 5 Schlüsseln.

Lighting

SRCALPHA-Dunkel-Surface, Taschenlampen-Kegel als Polygon (30 Segmente), Kerzen-Glows, 8 vorberechnete Filmkorn-Frames, Vignette.

SoundManager

Prozeduraler Horror-Ambient-Loop via NumPy, bodenabhängige Schrittgeräusche, Herzschlag beschleunigt im HUNT-Modus, Lautstärke-Steuerung.

ZoomCamera

Sanftes Zoom-Lerp zwischen ZOOM_MIN und ZOOM_MAX, verfolgt Spieler im aktuellen Raum, gibt view_size in Weltpixeln zurück.

PauseMenu

Lautstärke-Schieberegler für Musik und Sound, vollständige Maus- und Tastatursteuerung, Vollbild-Toggle, Rückkehr zum Hauptmenü.

JumpscareManager

Drei Jumpscare-Typen (Fast-Miss, raumspezifisch, Game-Over), Screen-Shake via ShakeOffset-Buffer, animierter Übergang mit Fade-Out.

— 06

KI-Verwendung

Für algorithmisch anspruchsvolle Stellen wurde gezielt **Claude Opus 4.5** als KI-Assistent verwendet. Alle KI-generierten Code-Abschnitte sind im Quellcode mit Markierungen gekennzeichnet:

```
# KI CODE ANFANG — Claude Opus 4.5# Prompt: "..."  
... # generierter Code# KI CODE ENDE
```

— VERWENDETE KI-STELLEN

#	MODUL	VERWENDUNGSZWECK & PROMPT-KURZFORM
01	lighting.py	8 Film-Grain-Surfaces mit NumPy (zufällige RGBA-Werte, transponiert)
02	lighting.py	Taschenlampen-Kegel als SRCALPHA-Polygon (30 Bogensegmente)
03	monster.py	BFS-Wegfindung für Monster-Navigation im 20×20px Grid
04	sound.py	24s Horror-Ambient-Loop (Sub-Bass 33Hz, Dissonanz 55/58Hz, Tremolo 110Hz)
05	main.py	Kamera-Zoom mit sanftem Lerp, begrenzt auf ZOOM_MIN/ZOOM_MAX
06	main.py	Pause-Menü mit Lautstärke-Schieberegler (Maus + Tastatur)

— WAS WIR SELBST ENTWICKELT HABEN

Der Großteil des Codes – Spielschleife, Raumsystem, Aufgabenlogik, HUD, Menüs, Highscore, Kollision, Steuerung und Spielbalancing – wurde vollständig eigenständig entwickelt. KI-generierte Stellen wurden stets verstanden, nachvollzogen und bei Bedarf angepasst.

— 07

Steuerung & Bedienung

— VOLLSTÄNDIGE TASTENBELEGUNG

TASTE / EINGABE	AKTION
W / A / S / D · ↑ ↓ ← →	Spieler bewegen
Maus (bewegen)	Taschenlampe ausrichten
F	Taschenlampe ein- / ausschalten
Shift	Schleichen — reduziert Lärmpegel stark
E	Interagieren (Aufgaben, Türen, Objekte)
Leertaste	Dash (60s Cooldown)
ESC	Pause-Menü öffnen
F11	Vollbild / Fenstermodus umschalten
H	Highscore-Tabelle anzeigen
R	Zurück zum Hauptmenü

— TIPPS & HINWEISE

- Batterien können erst aufgesammelt werden, wenn der Akku unter 15% sinkt.
- Der Code für das Kombinationsschloss steht auf einem Zettel in der Küche.
- Teppichböden im Schlafzimmer und Büro dämpfen Schrittgeräusche stark.

— SPIELSTART & INSTALLATION

```
# Abhängigkeiten installieren
pip install -r requirements.txt

# Spiel starten
cd src
python main.py
```

— 08

Reflexion & Fazit

— WAS WIR GELERNT HABEN

Das Projekt hat uns viel über objektorientierte Programmierung in Python beigebracht. Sobald man komplexere Themen wie Zoom, Kollision, Kamera oder Beleuchtung baut, merkt man wie viel Detailarbeit dahintersteckt – auch in einem vermeintlich einfachen Framework wie pygame.

— SCHWIERIGKEITEN

- SRCALPHA-Beleuchtung: transparent auf dunkel zeichnen war initial sehr verwirrend.
- BFS-Algorithmus: komplex – hier wurde KI-Unterstützung eingesetzt und der Code selbst nachvollzogen.
- Prozedurale Soundgenerierung mit NumPy: mehrere Frequenzschichten zu einem Horror-Track zu kombinieren.
- Kamera-Zoom mit sanftem Lerp: hat mehrere Iterationen bis zu einem natürlichen Gefühl gebraucht.
- Git / OneDrive-Synchronisation des Entwicklungsordners hat mehrfach für Chaos gesorgt.

— FAZIT

Nightwatch ist das umfangreichste Programmierprojekt, das wir bisher umgesetzt haben. Wir sind stolz darauf, dass das Spiel vollständig spielbar ist, eine echte Horror-Atmosphäre erzeugt und dabei komplett ohne externe Assets auskommt. KI haben wir für die schwierigsten algorithmischen Teile genutzt und diese klar im Code markiert – den Rest haben wir eigenständig entwickelt und verstanden.

"There is no escape. There is only dark."

— NIGHTWATCH, 2026

— QUELLENVERZEICHNIS

- [1] pygame 2.6.1 Dokumentation – <https://www.pygame.org/docs/>
- [2] NumPy Dokumentation – <https://numpy.org/doc/>
- [3] Pillow Dokumentation – <https://pillow.readthedocs.io/>
- [4] Python 3 Dokumentation – <https://docs.python.org/3/>
- [5] BFS-Algorithmus – Wikipedia: Breadth-first search